

CP301

Development Engineering Project

ML for Physicists

Mohi Modi
2023EPB1273

Supervisor –
Dr. Shankhadeep Chakrabortty

INDEX

- Motivation
- Objective
- Second Order Linear Differential Equation
- Neural Network for Differential Equations
- Neural Network Learning Framework
- Quantum Harmonic Oscillator
- Numerical method for QHO
- PINN formulation for QHO
- PINN v/s Analytical Solution
- Damped Harmonic Oscillator
- Trained v/s Untrained Data
- Role of Data in PINNs
- Singular Differential Equations
- Hyper-Geometric Equations
- Hypergeometric Equations Across Physics
- BTZ Black Hole Equation
- Future Scope
- Conclusion

MOTIVATION

- Mathematical description of physical systems are governed by differential equations
- Analytical solutions are often limited
- Numerical methods may become computationally expensive or unstable
- Machine learning offers an alternative framework to achieve approximated analytical form of solution
- We apply these methods from simple physical systems to black hole physics

OBJECTIVE

- Apply machine learning techniques to solve physical differential equations
- Compare neural network solutions with analytical and numerical methods
- Study how training and residual minimization improve physical accuracy
- Extend machine learning methods to singular differential equations
- Apply the framework to the BTZ black hole wave equation and analyze boundary behavior

Second Order Linear Differential Equation (S.O.L.D.E.)

A general second order differential equation has the form –

$$y'' + p(x)y' + q(x)y = 0$$

The general solution is given as –

$$y(x) = C_1 y_1(x) + C_2 y_2(x)$$

where,

$y_1(x)$ and $y_2(x)$ → two independent solutions

C_1 and C_2 → constant determined by boundary /initial conditions.

Neural Network for Differential Equations

A Neural Network (N.N.) learns a function $y_\theta(x)$.

- Input $\rightarrow x$
- Output $\rightarrow$ predicted $y(x)$

Using automatic differentiation, we compute the derivatives –

- $y'(x)$
- $y''(x)$

Then, a residual function is generated –

$$R(x) = y'' + p(x)y' + q(x)y$$

A loss function is built to train the model –

$$L = \langle \mathrm{R(x)}^2 \rangle$$

$$\Rightarrow L = (y'' + p(x)y' + q(x)y)^2$$

The N.N. is trained so that this loss becomes zero.

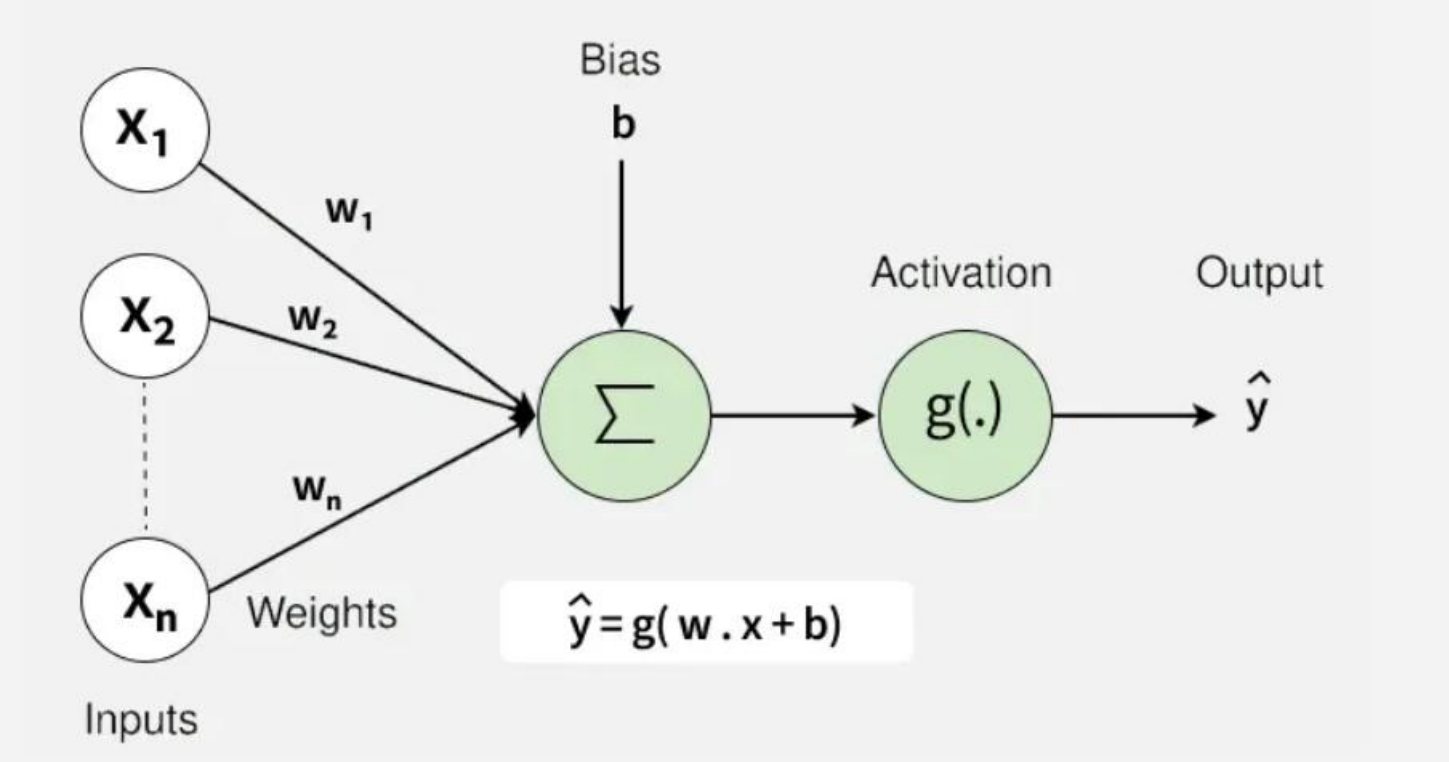
Neural Network Learning Framework

A Neural Network (N.N.) usually gives **one solution** only – $y(x)$.

This is **because** the network applies operations like – $y(x) = f(x; w, b)$ where,

- $w =$ weights
- $b =$ biases

So, for a set of weights, the network produces **one output for each input**. Thus, the network represents one function.



- To get two solutions for a S.O.D.E., we must **train the network twice** with different conditions.
- Numerical methods **apply ICs/BCs directly**; while PINNs enforce them through **loss minimization during training**.

Quantum Harmonic Oscillator

Consider the time independent QHO equation –

$$-\frac{\hbar^2}{2m}\frac{d^2\psi_i}{dx^2} + \frac{1}{2}m\omega^2x^2\psi_i = E\psi_i$$

The exact solution is given as a Hermite Polynomial –

$$\psi_n(x) = N_n H_n(x)\left(\sqrt{\frac{m\omega}{\hbar}}x\right)exp\left(-\frac{m\omega x^2}{2\hbar}\right)$$

where,

- N_n is the normalization constant –

$$N_n = \left(\frac{m\omega}{\pi\hbar}\right)^{1/4}\frac{1}{\sqrt{2^n n!}}$$

- $H_n(x)$ is the Hermite polynomial – few are shown below –

$$H_0(x) = 1$$
$$H_1(x) = 2x$$
$$H_2(x) = 4x^2 - 2$$
$$H_3(x) = 8x^3 - 12x$$
$$H_4(x) = 16x^4 - 48x^2 + 12$$

Numerical method to solve a Q.H.O.

Finite Difference Approximation

- Discretize space
  - Choose grid points
    $$x_0, x_1, \dots\dots\dots, x_N$$
  - Spacing
    $$h = x_{i+1} - x_i$$
- Second derivative is –
$$\frac{d^2\psi}{dx^2} = \frac{\psi_{(i+1)} - 2\psi_i + \psi_{(i-1)}}{h^2}$$

- Substitute into the Schrödinger equation –
$$\frac{\psi_{(i+1)} - 2\psi_i + \psi_{(i-1)}}{h^2} + x_i^2\psi_i = E\psi_i$$

- Rearrange –
$$-\frac{1}{h^2}\psi_{i+1} + \left(\frac{2}{h^2} + x_i^2\right)\psi_i - \frac{1}{h^2}\psi_{i-1} = E\psi_i$$

This becomes a matrix equation –

$$H\psi = E\psi$$

The finite difference method **converts** the Schrödinger differential equation into a **matrix eigenvalue problem**. Solving this matrix numerically gives the energy levels and wavefunctions of the Quantum Harmonic Oscillator.

PINN formulation for Q.H.O.

The N.N. approximates a solution - $\psi_\theta(x)$

- Input $\longrightarrow x$
- Output $\longrightarrow$ predicted wavefunction

Using automatic differentiation we compute –

- $\psi'(x)$
- $\psi''(x)$

Residual of the Schrödinger equation –

$$R(x) = \psi'' + (2\epsilon - x^2)\psi$$

Loss –

$$L = \langle R(x)^2 \rangle + L_{IC}$$

The network adjusts the weights to minimize this loss, so that the predicted $\psi(x)$ satisfies the Schrödinger equation.

For a QHO, the wave function should vanish at $x \to \infty$, i.e.,

$$\psi(x \to \infty) \to 0$$

Consider a network of 2 layers.

⟶ First layer computation

- Input ⟶ x
- Linear operation

$$z_1 = w_1 x + b_1$$

- Activation function

$$a_1 = \tanh(z_1)$$

This becomes the output of the first layer.

The output of the first layer is the input to the second layer –

- Input → a_1
- Linear operation

$$z_2 = w_2 a_1 + b_2$$

- Generally, the last layer of PINN is linear

$$a_2 = z_2$$

Thus, the predicted wavefunction is –

$$\boldsymbol{\psi(x) = a_2 = z_2}$$

QHO solutions obtained using PINN

```
Learned Energy: 0.6596318483352661

--- Solution 1 ---
```

$$H_7(x) = -0.01x^4 + 0.02x^3 + 0.02x^2 - 0.03x + 0.98$$

$$\psi_1(x) = (-0.01x^4 + 0.02x^3 + 0.02x^2 - 0.03x + 0.98)e^{-x^2/2}$$

```
---Solution 2 ---
```

$$H_7(x) = -0.02x^3 + 0.73x$$

$$\psi_2(x) = (-0.02x^3 + 0.73x)e^{-x^2/2}$$

PINN v/s Analytical Solution

- Neural Networks **learn a function $u_\theta(x)$** using its weights and biases.
- Outputs can be evaluated at any point.
- The learned function **can be reconstructed** using

  interpolation; curve fitting; symbolic regression

  but it **may not be the exact** simple analytical form.

Damped Harmonic Oscillator

Equation of a damped harmonic oscillator –

$$m\frac{d^2x}{dt^2} + c\frac{dx}{dt} + kx = 0$$

$$\zeta = \frac{c}{2\sqrt{mk}}$$

where,

- $\boldsymbol{m}$: mass
- $\boldsymbol{c}$: damping coefficient
- $\boldsymbol{k}$: spring constant
- $\boldsymbol{\zeta}$: damping ratio

ζ	Type of motion	Behaviour
0	No damping	Pure oscillation
(0, 1)	Underdamped	Oscillations slowly decay
1	Critically damped	Fastest return to equilibrium
> 1	Overdamped	No oscillation, slow return

Considering – $\mathbf{m} = 1$, $\mathbf{c} = 0.4$, $\mathbf{k} = 1$

$$\Rightarrow \boldsymbol{\zeta} = 0.2$$

Thus, we consider an underdamped system.

Initial conditions –

$$x(0) = 1$$
$$x'(0) = 0$$

Analytical Solution

Assume an exponential solution

$$x(t) = e^{rt}$$

Compute the derivatives and substitute into the differential equation –

$$\Rightarrow e^{rt}(r^2 + 0.4r + 1) = 0$$

$\because e^{rt} \neq 0$

$$\therefore \boldsymbol{r^2 + 0.4r + 1 = 0} \qquad \textit{(characteristic equation)}$$

$$\boldsymbol{x(t) = e^{-0.2t}[cos\, 0.98t + 0.204\, sin\, 0.98t]}$$

Trained v/s Untrained Neural Network

t (s)	Untrained		Trained	
	x(t)	Residual	x(t)	Residual
0.00	0.1422	-0.0520	1.000000	0.000000
0.25	0.1318	-0.0776	0.969389	0.000000
0.50	0.1092	-0.1015	0.881457	0.000000
0.75	0.0749	-0.1141	0.744364	0.000000
1.00	0.0310	-0.1062	0.568797	0.000000

- Residual increases with time
- Predictions appear smooth but are inaccurate
- No optimization has occurred

After training:

- $x(t)$ decreases → system moves toward equilibrium
- velocity increases initially → restoring force dominates
- acceleration remains negative → force acts toward equilibrium
- residual approaches zero → governing equation is satisfied

Training significantly improved physical consistency.

Role of Data in PINNs

PINNs can work **without labelled datasets** because they learn using:

- Governing differential equations
- Initial conditions
- Boundary conditions

These physical constraints act as supervision.

Data becomes necessary when:

- governing equations are incomplete
- experimental noise exists
- parameters are unknown
- exact solutions are unavailable

PINNs combine physics-driven learning with data-driven learning whenever required.

Hypergeometric Equation

Standard form –

$$x(1-x)\frac{d^2y}{dx^2} + [c-(a+b+1)x]\frac{dy}{dx} - aby = 0$$

here, $y(x) \rightarrow$ unknown function

$a, b, c \rightarrow$ constants or parameters

This equation contains **regular singular points at**

$$x = 0,\ 1,\ \infty$$

Near these points:

- solutions may behave differently
- standard methods become difficult
- Frobenius/hypergeometric methods are used

Derivation

Assume a series solution – Frobenius form

$$y = \sum_{n=0}^{\infty} a_n x^{n+r}$$

Substitution gives indicial roots – $\boldsymbol{r = 0}$ & $\boldsymbol{r = 1 - c}$

Recurrence Relation –

$$\therefore a_n = \frac{(a)_n (b)_n}{(c)_n n!} a_0$$

⟶ Final Solution: $r = 0 \rightarrow \boldsymbol{y = {}_2F_1(a, b; c; x)}$

$$r = 1 - c \rightarrow \boldsymbol{y = x^{1-c} {}_2F_1(a - c + 1, b - c + 1; 2 - c; x)}$$

Hypergeometric Equations Across Physics

- **Classical Physics** – Laplace's equation in electrostatics leads to Legendre functions.
- **Quantum Mechanics** – Hydrogen atom wave equations lead to Laguerre functions.
- **Statistical Physics** – Ising model correlation functions involve hypergeometric functions.
- **Black Hole Physics** – BTZ black hole radial equations reduce to hypergeometric form.
- **Cosmology / Astrophysics** – Inflation perturbation equations can have hypergeometric solutions.
- **Condensed Matter Physics** – Graphene electron dynamics can reduce to hypergeometric equations.

BTZ Black Hole Equation

The BTZ Black Hole is a solution to Einstein's equations in (2+1) dimensions.

Transformation to Hypergeometric Form

In this equation, when we separate variables –

$$\Phi(t, r, \phi) = e^{-\iota(\omega t - m\phi)} R(r)$$

The radial part changes to **hypergeometric equation** –

$$\boldsymbol{x(1-x)\frac{d^2y}{dx^2} + [c - (a+b+1)x]\frac{dy}{dx} - aby = 0}$$

This ODE is a representation of a wave propagation in curved spacetime.

Singularity lies at –

$$z = 0, 1, \infty$$

Physical points or boundaries are considered at –

$$z = 0 \rightarrow event\ horizon \rightarrow point\ of\ no\ return$$

$$z = 1 \rightarrow AdS\ boundary \rightarrow very\ far\ away\ region$$

Position	Type of solution	Mathematical form	Interpretation
$z = 0$	Regular	$R(z) \sim 1$	finite, smooth, no divergence
	Singular	$R(z) \approx z^{1-c}$	incoming and outgoing modes
$z = 1$	Regular	$R(z) \sim constant$	finite, stable at infinity
	Singular	$R(z) \sim (1-z)^{c-a-b}$	how field behaves at large distance

Results

BTZ equation and solutions

BTZ Hypergeometric Equation –

$$-abR(z) + z(1-z)\frac{d^2}{dz^2}R(z) + \big(c - z(a+b+1)\big)\frac{d}{dz}R(z) = 0$$

General Analytic Solutions

$$R_0(z) = {}_2F_1\left(\begin{matrix}a, b\\ c\end{matrix}\middle| z\right)$$

$$S_0(z) = z^{1-c}\,{}_2F_1\left(\begin{matrix}a-c+1, b-c+1\\ 2-c\end{matrix}\middle| z\right)$$

$$R_1(z) = {}_2F_1\left(\begin{matrix}a, b\\ a+b-c+1\end{matrix}\middle| 1-z\right)$$

$$S_1(z) = (1-z)^{-a-b+c}\,{}_2F_1\left(\begin{matrix}-a+c, -b+c\\ -a-b+c+1\end{matrix}\middle| 1-z\right)$$

Numerical Analytic Solutions ($\mathbf{a = 0.5,\ b = 0.7,\ c = 1.6}$)

$$R_0(z) = {}_2F_1\left(\begin{matrix}0.5, 0.7\\ 1.6\end{matrix}\middle| z\right)$$

$$S_0(z) = \frac{{}_2F_1\left(\begin{matrix}-0.1, 0.0999999999999999\\ 0.4\end{matrix}\middle| z\right)}{z^{0.6}}$$

$$R_1(z) = {}_2F_1\left(\begin{matrix}0.5, 0.7\\ 0.6\end{matrix}\middle| 1-z\right)$$

$$S_1(z) = (1-z)^{0.4}\,{}_2F_1\left(\begin{matrix}0.9, 1.1\\ 1.4\end{matrix}\middle| 1-z\right)$$

PINN Derived Approximate Basis near z = 0

$$R(z=0) \approx -0.442634z^4 + 0.402964z^3 + 0.0457673z^2 + 0.219099z + 0.999943$$

$$S(z=0) \approx \frac{-0.034955z^4 + 0.0912184z^3 - 0.0780418z^2 + 0.000784482z + 0.999614}{z^{0.6}}$$

PINN Derived Approximate General Solution near z = 0

$$R_0(z) \approx \frac{C_1 z^{0.6}\left(-0.442634z^4 + 0.402964z^3 + 0.0457673z^2 + 0.219099z + 0.999943\right) + C_2\left(-0.034955z^4 + 0.0912184z^3 - 0.0780418z^2 + 0.000784482z + 0.999614\right)}{z^{0.6}}$$

PINN Derived Approximate Basis near z = 1

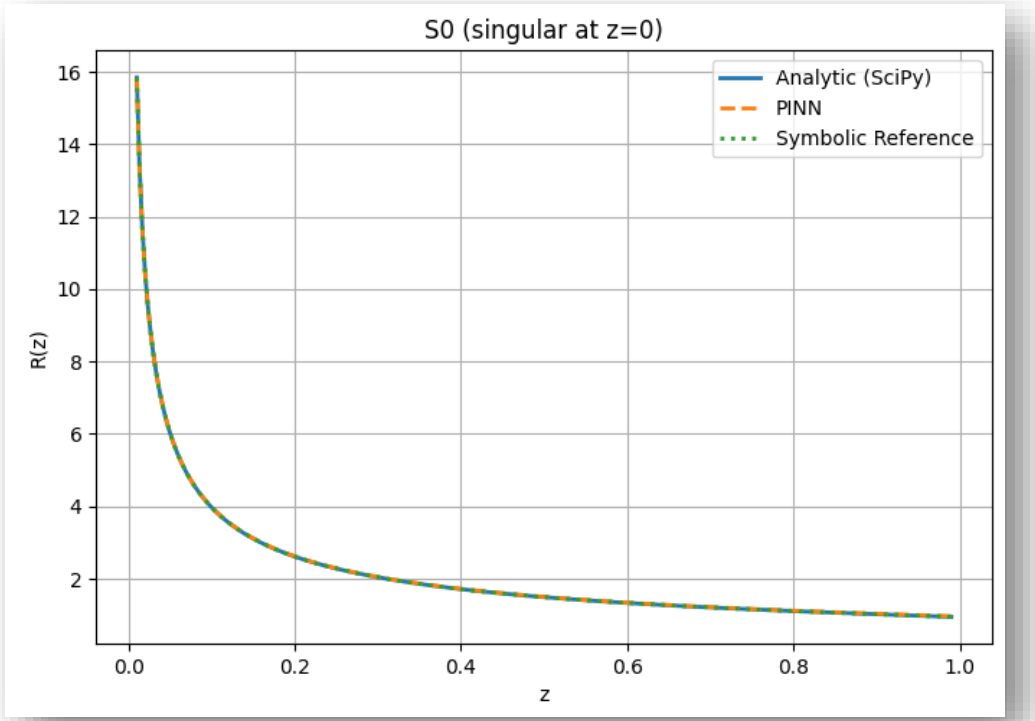
$$R(z=1) \approx -0.584534z - 0.161071\,(1-z)^4 + 0.792656\,(1-z)^3 + 0.389427\,(1-z)^2 + 1.58444$$

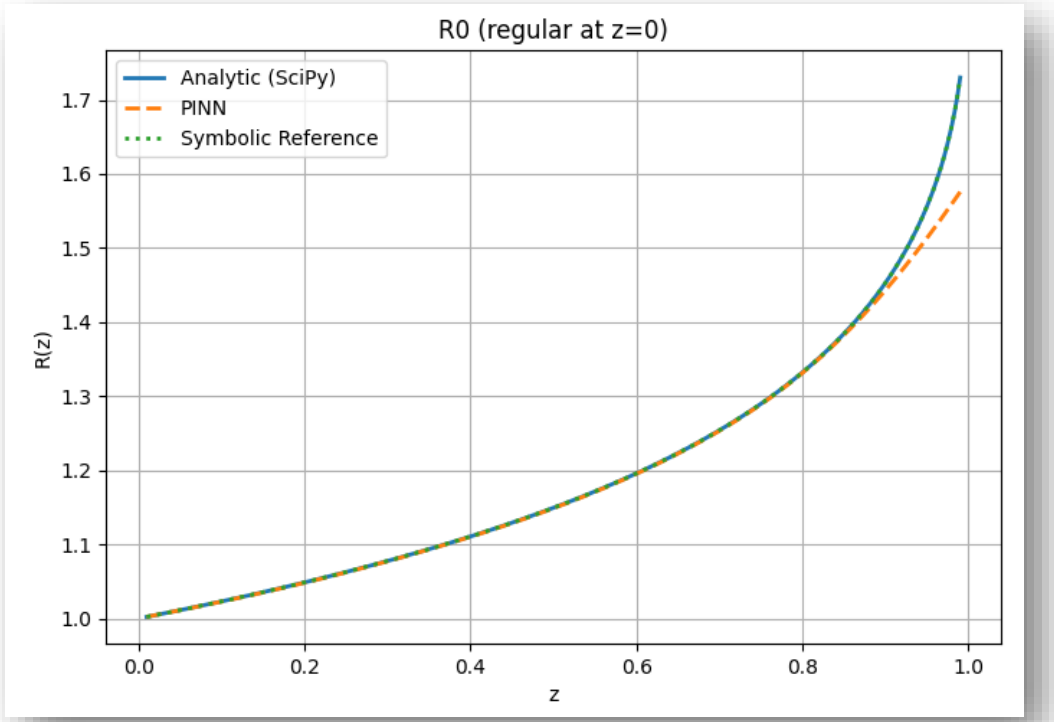
$$S(z=1) \approx (1-z)^{0.4}\left(-0.707214z + 0.623211\,(1-z)^4 + 0.5449\,(1-z)^3 + 0.579158\,(1-z)^2 + 1.70714\right)$$

PINN Derived Approximate General Solution near z = 1

$$R_1(z) \approx -C_3\left(0.584534z + 0.161071\,(z-1)^4 + 0.792656\,(z-1)^3 - 0.389427\,(z-1)^2 - 1.58444\right) + C_4\,(1-z)^{0.4}\left(-0.707214z + 0.623211\,(z-1)^4 - 0.5449\,(z-1)^3 + 0.579158\,(z-1)^2 + 1.70714\right)$$

Inference near $z = 0$





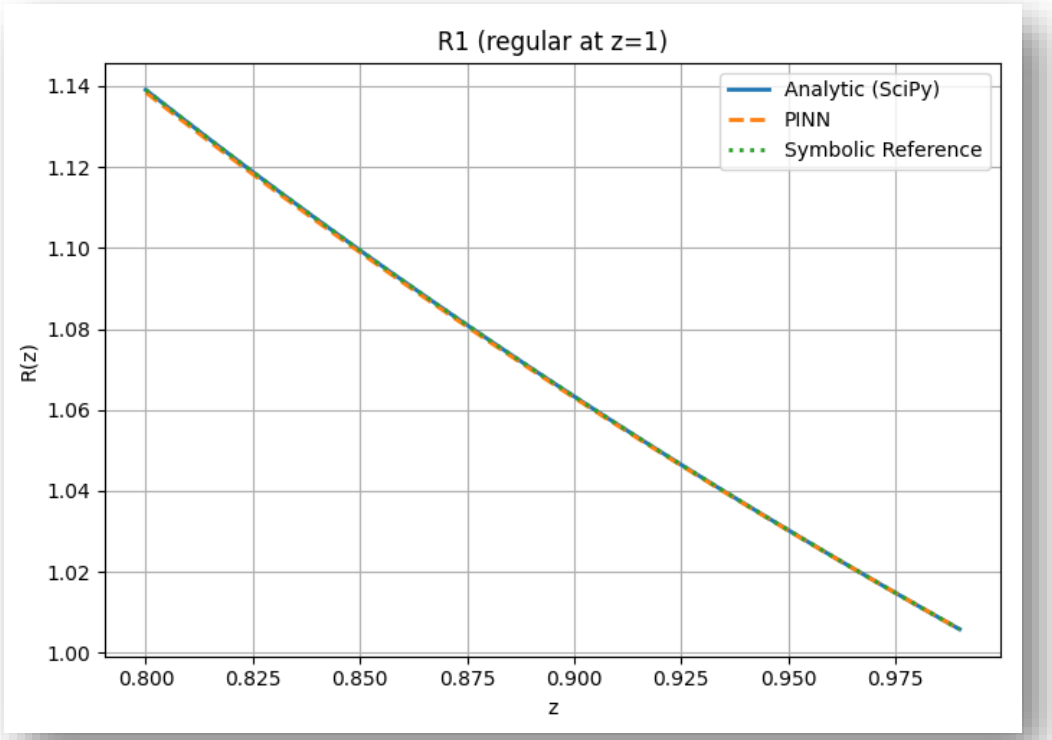
PINN closely matches the analytical and symbolic solutions near $z = 0$, showing that the model learned the local behavior correctly.

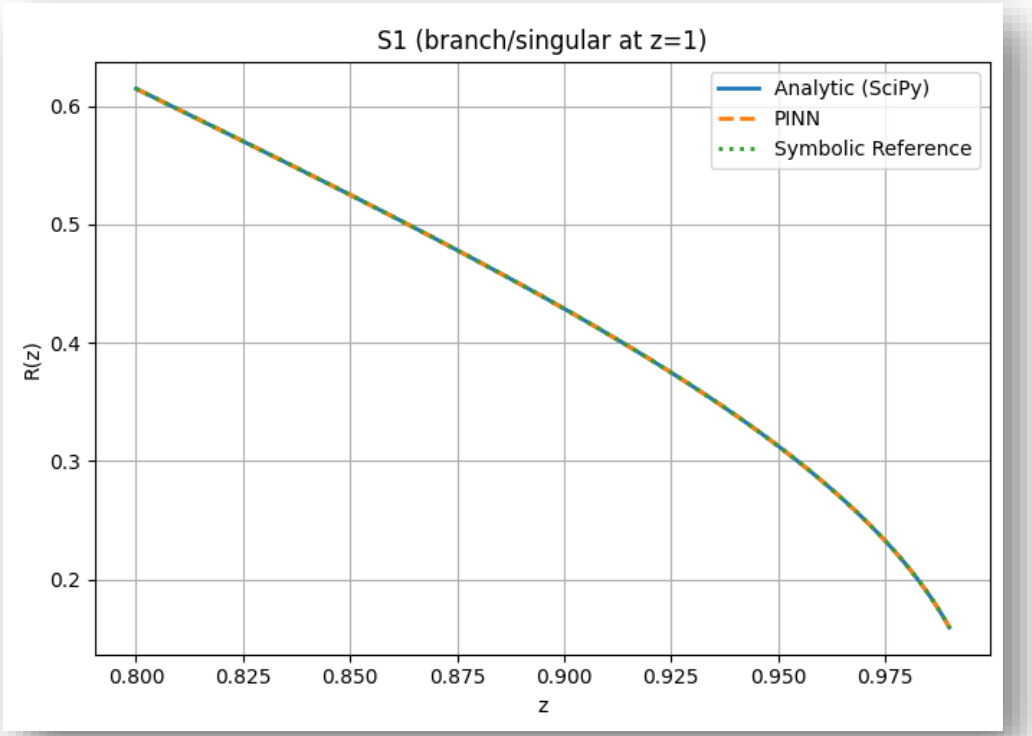
- The singular solution S_0 shows the expected divergence:

$$S_0(z) \sim z^{-0.6}$$

- This confirms that PINN can handle the singular nature of the BTZ hypergeometric equation near $z = 0$.

Inference near $z = 1$





PINN overlaps very well with analytical and symbolic solutions near $z = 1$, confirming accurate learning in the second local region.

- The branch solution S_1 follows the expected behavior:

$$S_1(z) \sim (1 - z)^{0.4}$$

- This shows that PINN can also reproduce the correct boundary behavior near $z = 1$, where the BTZ equation has another singular point.

Future Scope: Beyond BTZ

- BTZ black hole was used as a benchmark because analytical hypergeometric solutions are known.
- In higher-dimensional black holes like AdS-Schwarzschild black hole (5D), analytical connection formulas are often unavailable.
- Existing studies largely rely on numerical methods for these systems.
- PINNs may help obtain global solutions and infer connection behavior between boundaries.

Conclusion

- Applied machine learning to solve physics-based differential equations
- Validated results using **QHO** and **DHO**
- Enforced physical laws through equations, boundary conditions, and residual minimization
- Extended the study to **singular differential equations**
- Transformed the **BTZ wave equation** into hypergeometric form to study boundary behavior
- Demonstrated ML applications in **quantum, classical and gravitational physics**

THANK YOU!